Práctica guiada de control de versiones con Git, GitHub e IntelliJ IDEA

Al terminar la práctica deberás ser capaz de:

- crear un proyecto Java desde cero en IntelliJ,
- inicializar un repositorio Git local,
- conectar ese repositorio con GitHub,
- realizar commits con sentido,
- crear y cambiar entre ramas,
- subir ramas a remoto,
- fusionarlas,
- resolver conflictos de fusión,
- verificar el estado del proyecto en local y en GitHub,
- interpretar el historial visual de ramas y commits.

Vamos a desarrollar una pequeña aplicación Java de consola llamada:

Gestor de Tareas

Git registra la evolución del proyecto.

0. Requisitos previos

- IntelliJ IDEA instalado
- Git instalado
- una cuenta de GitHub
- conexión a Internet
- una carpeta de trabajo local donde guardar el proyecto

Opcional, pero recomendable:

- haber iniciado sesión en GitHub desde IntelliJ

1. Creación del proyecto Java en IntelliJ

Qué vamos a hacer

Crear un proyecto Java vacío y añadir tres clases básicas.

Acción en IntelliJ

1. Abrir IntelliJ IDEA.
2. Pulsar New Project.
3. Elegir Java.
4. Comprobar que hay un JDK seleccionado.
5. Nombre del proyecto: PracticaGitIntelliJ
6. Elegir ubicación.
7. Pulsar Create.

Qué debe verse

Se crea un proyecto nuevo con su estructura mínima.

2. Crear las clases del proyecto

Vamos a crear tres clases:

- Main
- Tarea
- GestorTareas

2.1 Crear Main.java

Dentro de src, crear la clase Main.

Código inicial:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Aplicación Gestor de Tareas");
    }
}
```

2.2 Crear Tarea.java

```java
public class Tarea {
    private String titulo;
    private boolean completada;

    public Tarea(String titulo, boolean completada) {
        this.titulo = titulo;
        this.completada = completada;
    }

    public String getTitulo() {
        return titulo;
    }

    public boolean isCompletada() {
        return completada;
    }

    @Override
    public String toString() {
        return titulo + " - " + (completada ? "Completada" : "Pendiente");
    }
}
```

2.3 Crear GestorTareas.java

```java
import java.util.ArrayList;
import java.util.List;

public class GestorTareas {
    private List<Tarea> tareas = new ArrayList<>();

    public void agregarTarea(Tarea tarea) {
        tareas.add(tarea);
    }

    public void listarTareas() {
        if (tareas.isEmpty()) {
            System.out.println("No hay tareas.");
        } else {
            for (Tarea tarea : tareas) {
                System.out.println(tarea);
            }
        }
    }
}
```

3. **Ejecutar el proyecto por primera vez**

   Todavía no hay Git. Puede ser que se haya generado de forma automática al crear el proyecto
   Solo tenemos un proyecto Java normal. Si ya estamos vinculados, pedirá cada vez que creemos un .java si lo queremos vincular, diremos que sí.

4. **Inicializar Git en el proyecto**

   Convertimos el proyecto en un repositorio Git local. Si no se hubiera hecho en principio.
   Acción en IntelliJ

   1. Menú VCS (o git directamente en versiones modernas)
   2. Enable Version Control Integration…
   3. Seleccionar Git
   4. Aceptar

   Qué pasa realmente
   IntelliJ crea internamente la carpeta **.git.**
   Comando Git equivalente

   git init

Los archivos empiezan a aparecer como controlados o pendientes de commit.
A partir de ahora:

- Git observa cambios,
- pero todavía no hay ningún commit,
- y todavía no hay GitHub.

5. **Crear el archivo .gitignore**

Evitamos subir archivos propios del IDE o compilados.
Crear en la raíz del proyecto un archivo llamado **.gitignore** con este contenido:

```
.idea/
*.iml
out/
```

Puede que ya se haya creado, en ese caso debemos abrirla e incluir esto si no estuviera.
No todo lo que hay en la carpeta del proyecto debe versionarse.
Git debe seguir:

- código fuente,
- recursos del proyecto,
- configuración necesaria.

Pero no debe seguir:

- archivos temporales,
- configuración local irrelevante,
- compilados.

6. **Comprobar estado de Git**

Acción en IntelliJ
Abrir la ventana de cambios o la pestaña de Git.
Comando Git equivalente

```
git status
```

Qué deberíamos tener como archivos nuevos

- `.gitignore`
- `src/Main.java`
- `src/Tarea.java`
- `src/GestorTareas.java`

7. **Primer commit local**

Guardamos el primer estado del proyecto.
Acción en IntelliJ

1. Abrir la ventana Commit
2. Seleccionar todos los archivos
3. Escribir el mensaje:
   `Creación inicial del proyecto GestorTareas`
4. Pulsar Commit

Comando Git equivalente

```
git add .
git commit -m "Creación inicial del proyecto GestorTareas"
```

Este commit:

- existe solo en local,
- no está aún en GitHub,
- es el primer punto estable del historial.

8. **Crear el repositorio remoto en GitHub**

Creamos el repositorio remoto vacío.
Acción en GitHub

1. Entrar en GitHub
2. New repository
3. Nombre: **practica-git-intellij**

4. Público o privado
5. Muy importante: no marcar README, ni .gitignore, ni licencia
6. Crear repositorio

Por qué debe estar vacío

Porque el proyecto ya existe en local y no queremos que GitHub genere una historia inicial distinta.

9. Conectar el repositorio local con GitHub

Puedes hacerlo desde IntelliJ o desde terminal.

9.1 Opción desde IntelliJ

Usar la opción de compartir proyecto en GitHub o añadir remoto desde configuración Git.

Obtener URL del repositorio remoto

En GitHub:

- Botón **Code**
- Copiar URL HTTPS

Ejemplo:

`https://github.com/usuario/repositorio.git`

Añadir repositorio remoto en IntelliJ

Ruta:

Git → Manage Remotes

Acciones:

- Pulsar +
- Name: origin
- URL: (pegar URL de GitHub)
- Confirmar con **OK → OK**

Verificar configuración

En IntelliJ:

- Volver a **Manage Remotes**
- Debe aparecer: `origin → https://github.com/usuario/repositorio.git`

O por consola: `git remote -v`

Subir el proyecto (push)

Ruta: `Git → Push`

Si es la primera vez:

Aceptar **Set upstream**

Comprobación final

- Acceder al repositorio en GitHub. Actualizar (AltGr + F5)
- Verificar que los archivos se han subido correctamente

9.2 Opción por terminal

Situados en la carpeta raíz del proyecto, ejecutar:

```
git remote add origin https://github.com/USUARIO/practica-git-intellij.git
```

Sustituyendo USUARIO por el nombre real.

Después:

```
git branch -M main
git push -u origin main
```

Qué hace cada comando

`git remote add origin ...` Define el repositorio remoto llamado origin.

`git branch -M main` Renombra la rama actual a main.

`git push -u origin main` Sube la rama main al remoto y establece seguimiento.

Qué debe verse en GitHub

Deben aparecer:

- los tres .java
- el .gitignore
- la rama main

10. **Primer cambio en main**

Vamos a modificar el proyecto en la rama principal antes de empezar a ramificar.

Modificar Main.java y dejarlo así:

```
public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();
        System.out.println("=== GESTOR DE TAREAS ===");
        gestor.listarTareas();
    }
}
```

Ejecutar el programa.

Salida esperada:

```
=== GESTOR DE TAREAS ===
    No hay tareas.
```

Hacer **commit** y después hacer **push** (no usar commit & push).

Mensaje:

```
Integración inicial de GestorTareas en Main
```

Comandos equivalentes

```
git add .
git commit -m "Integración inicial de GestorTareas en Main"
git push
```

11. **Ver el historial**

Acción en IntelliJ

Abrir Git Log.

Comando equivalente

```
git log --oneline --graph --all
```

Qué debes ver

Dos commits:

- creación inicial del proyecto
- integración de GestorTareas en Main

El historial ya cuenta la historia del proyecto.

12. **Crear la primera rama: menú de consola**

Abrimos una línea de desarrollo separada para crear el menú.

Acción en IntelliJ

1. Ir al selector de ramas
2. Crear nueva rama
3. Nombre: feature/menu-consola
4. Cambiar a esa rama

Comando equivalente

```
git checkout -b feature/menu-consola
```

Qué comprobar

La rama actual ya no es main, sino feature/menu-consola.

Comando útil

```
git branch
```

La rama actual aparecerá con asterisco.

13. **Desarrollo en la rama feature/menu-consola**

Sustituir Main.java por esto:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();
        Scanner sc = new Scanner(System.in);
        int opcion;
```

```
        do {
            System.out.println("=== GESTOR DE TAREAS ===");
            System.out.println("1. Listar tareas");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    gestor.listarTareas();
                    break;
                case 0:
                    System.out.println("Fin del programa");
                    break;
                default:
                    System.out.println("Opción no válida");
            }
        } while (opcion != 0);
    }
}
```

Ejecutar para comprobar funcionamiento.

Hacer **commit.**

Mensaje:

```
Añadido menú básico de consola
```

Comandos equivalentes

```
git add .
git commit -m "Añadido menú básico de consola"
```

Subir la rama a GitHub (**git -> push**, intellij lo detecta directamente) o en consola:

```
git push -u origin feature/menu-consola
```

Ahora hay una rama remota adicional.

En GitHub ya puede verse feature/menu-consola.

14. **Volver a main**

Cambiamos de rama para crear otra línea independiente de desarrollo.

Acción en IntelliJ

Cambiar a main.

Comando equivalente

```
git checkout main
```

Cuando cambiamos de rama, el contenido del proyecto puede cambiar en disco.

Cada rama apunta a un estado distinto del repositorio.

15. **Crear la segunda rama: tareas de ejemplo**

Crear rama **feature/tareas-ejemplo.**

Comando equivalente

```
git checkout -b feature/tareas-ejemplo
```

16. **Desarrollo en feature/tareas-ejemplo**

Modificar Main.java así:

```
public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();

        System.out.println("=== GESTOR DE TAREAS ===");

        gestor.agregarTarea(new Tarea("Preparar examen", false));
        gestor.agregarTarea(new Tarea("Corregir prácticas", true));
        gestor.agregarTarea(new Tarea("Subir materiales al aula virtual",
false));
        gestor.listarTareas();
    }
}
```

Ejecutar el programa.

Salida esperada:

```
=== GESTOR DE TAREAS ===
Preparar examen - Pendiente
Corregir prácticas - Completada
Subir materiales al aula virtual - Pendiente
```

Hacer commit.

Mensaje:

```
Añadidas tareas de ejemplo al inicio de la aplicación
```

Comandos equivalentes

```
git add .
git commit -m "Añadidas tareas de ejemplo al inicio de la aplicación"
git push -u origin feature/tareas-ejemplo
```

Ahora tenemos dos ramas que han modificado el mismo archivo, Main.java, pero de formas distintas.

Eso prepara un conflicto futuro.

17. **Volver a main otra vez**

```
git checkout main
```

18. **Crear la tercera rama: validación**

```
git checkout -b feature/validacion
```

Modificar **GestorTareas.java** así:

```
import java.util.ArrayList;
import java.util.List;

public class GestorTareas {
    private List<Tarea> tareas = new ArrayList<>();

    public void agregarTarea(Tarea tarea) {
        if (tarea == null) {
            System.out.println("No se puede agregar una tarea nula.");
            return;
        }
        tareas.add(tarea);
    }

    public void listarTareas() {
        if (tareas.isEmpty()) {
            System.out.println("No hay tareas.");
        } else {
            for (Tarea tarea : tareas) {
                System.out.println(tarea);
            }
        }
    }
}
```

Commit:

```
Añadida validación al agregar tareas
```

Comandos:

```
git add .
git commit -m "Añadida validación al agregar tareas"
git push -u origin feature/validacion
```

Esta rama toca otro archivo.

Por tanto, es candidata a fusionarse sin conflicto.

19. **Volver a main**

```
git checkout main
```

20. **Crear la cuarta rama: cambio de cabecera**

```
git checkout -b feature/cambio-cabecera
```

Modificar Main.java así:

```
public class Main {
    public static void main(String[] args) {
```

```
            GestorTareas gestor = new GestorTareas();
            System.out.println("=== ORGANIZADOR PERSONAL DE TAREAS ===");
            gestor.listarTareas();
        }
    }
```

Commit:

```
Actualizada la cabecera principal de la aplicación
```

Comandos:

```
git add .
git commit -m "Actualizada la cabecera principal de la aplicación"
git push -u origin feature/cambio-cabecera
```

21. **Revisar todas las ramas creadas**

Comando:

```
git branch
git branch -r
```

Qué deben verse

Locales:

- **main**
- **feature/menu-consola**
- **feature/tareas-ejemplo**
- **feature/validacion**
- **feature/cambio-cabecera**

Remotas:

- **origin/main**
- **origin/feature/menu-consola**
- **origin/feature/tareas-ejemplo**
- **origin/feature/validacion**
- **origin/feature/cambio-cabecera**

En IntelliJ también deben verse en el panel de ramas.

22. **Primer merge: rama sin conflicto**

Volvemos a main:

```
git checkout main
```

Fusionamos feature/validacion:

```
git merge feature/validacion
```

Qué debería pasar

Merge limpio, sin conflicto.

Por qué

Porque modifica GestorTareas.java, no compite con cambios previos en main.

Subimos el resultado:

```
git push
```

No todas las fusiones generan conflicto.

Git puede combinar automáticamente cambios independientes.

23. **Ver el grafo de historial**

```
git log --oneline --graph --all
```

Aquí ya debería empezar a verse una bifurcación y su integración.

24. **Fusionar la rama del menú**

Seguimos en main.

```
git merge feature/menu-consola
```

En esta práctica normalmente se fusionará bien si main aún no ha cambiado esa zona exacta de Main.java.

Si el merge se completa:

- hacer push
- verificar

```
git push
```

Qué se obtiene ahora en main

Un Main.java con menú, pero sin tareas de ejemplo y quizá sin cabecera nueva.

25. Intentar fusionar feature/tareas-ejemplo

Ahora viene el conflicto importante.

```
git merge feature/tareas-ejemplo
```

Resultado esperado

Git detectará conflicto en Main.java.

Mensaje típico

```
CONFLICT (content): Merge conflict in src/Main.java
Automatic merge failed; fix conflicts and then commit the result.
```

Esto no es un fallo del repositorio.

Es una situación normal: dos ramas han modificado de manera incompatible la misma parte del mismo archivo.

26. Ver el conflicto

Comando útil:

```
git status
```

Debemos ver algo como:

```
both modified: src/Main.java
```

En IntelliJ aparecerá el archivo como conflictivo y ofrecerá el editor de merge.

27. Abrir el editor de conflictos en IntelliJ

Abrir el conflicto con la herramienta de resolución visual.

Normalmente se verá:

- una versión a la izquierda,
- otra a la derecha,
- el resultado final en el centro o abajo, según interfaz.

No se trata de elegir "izquierda" o "derecha" sin pensar.

Hay que construir una versión integrada.

28. Resolver el conflicto manualmente

El archivo Main.java final debe quedar así:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();

        gestor.agregarTarea(new Tarea("Preparar examen", false));
        gestor.agregarTarea(new Tarea("Corregir prácticas", true));
        gestor.agregarTarea(new Tarea("Subir materiales al aula virtual",
false));

        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("=== GESTOR DE TAREAS ===");
            System.out.println("1. Listar tareas");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    gestor.listarTareas();
                    break;
                case 0:
                    System.out.println("Fin del programa");
                    break;
```

```
                    default:
                        System.out.println("Opción no válida");
                }
            } while (opcion != 0);
        }
    }
```

Qué se ha integrado aquí

- la creación de tareas de ejemplo,
- el menú de consola.
- Aplicamos cambios y marcamos como resuelto.

29. Marcar el conflicto como resuelto

En IntelliJ, aceptar la resolución.

Si se hace por comandos:

```
git add src/Main.java
```

Después completar el merge:

```
git commit -m "Resuelto conflicto entre menú y carga inicial de tareas"
```

Y subirlo:

```
git push
```

30. Intentar fusionar la rama de cabecera

Ahora aún falta integrar feature/cambio-cabecera.

```
git merge feature/cambio-cabecera
```

Qué puede pasar

Dos posibilidades razonables:

1. Merge automático si Git interpreta que el cambio puede incorporarse.
2. Conflicto menor sobre Main.java.

31. Resultado final deseado

Queremos que la cabecera final sea esta:

```
System.out.println("=== ORGANIZADOR PERSONAL DE TAREAS ===");
```

Por tanto, si tras el merge sigue apareciendo:

```
System.out.println("=== GESTOR DE TAREAS ===");
```

Debemos corregir manualmente el archivo.

La versión final completa de Main.java debe quedar así:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();

        gestor.agregarTarea(new Tarea("Preparar examen", false));
        gestor.agregarTarea(new Tarea("Corregir prácticas", true));
        gestor.agregarTarea(new Tarea("Subir materiales al aula virtual",
false));

        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("=== ORGANIZADOR PERSONAL DE TAREAS ===");
            System.out.println("1. Listar tareas");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    gestor.listarTareas();
                    break;
                case 0:
```

```
                    System.out.println("Fin del programa");
                    break;
                default:
                    System.out.println("Opción no válida");
            }
        } while (opcion != 0);
    }
}
```

Si ha habido conflicto:

```
git add src/Main.java
git commit -m "Integrada cabecera final en main"
git push
```

32. Comprobación funcional final

Ejecutar la aplicación.

Comportamiento esperado

Al arrancar:

- se cargan las tareas de ejemplo,
- aparece el menú,
- al elegir 1 se listan las tareas,
- al elegir 0 se sale.

33. Estado final de las clases

Tarea.java

```
public class Tarea {
    private String titulo;
    private boolean completada;

    public Tarea(String titulo, boolean completada) {
        this.titulo = titulo;
        this.completada = completada;
    }

    public String getTitulo() {
        return titulo;
    }

    public boolean isCompletada() {
        return completada;
    }

    @Override
    public String toString() {
        return titulo + " - " + (completada ? "Completada" : "Pendiente");
    }
}
```

GestorTareas.java

```
import java.util.ArrayList;
import java.util.List;

public class GestorTareas {
    private List<Tarea> tareas = new ArrayList<>();

    public void agregarTarea(Tarea tarea) {
        if (tarea == null) {
            System.out.println("No se puede agregar una tarea nula.");
            return;
        }
        tareas.add(tarea);
    }
```

```
    public void listarTareas() {
        if (tareas.isEmpty()) {
            System.out.println("No hay tareas.");
        } else {
            for (Tarea tarea : tareas) {
                System.out.println(tarea);
            }
        }
    }
}
```

Main.java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();

        gestor.agregarTarea(new Tarea("Preparar examen", false));
        gestor.agregarTarea(new Tarea("Corregir prácticas", true));
        gestor.agregarTarea(new Tarea("Subir materiales al aula virtual",
false));

        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("=== ORGANIZADOR PERSONAL DE TAREAS ===");
            System.out.println("1. Listar tareas");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    gestor.listarTareas();
                    break;
                case 0:
                    System.out.println("Fin del programa");
                    break;
                default:
                    System.out.println("Opción no válida");
            }
        } while (opcion != 0);
    }
}
```

34. Qué deben revisar en IntelliJ al finalizar

En IntelliJ deben comprobar cuatro zonas:

- **Primera: el proyecto** -> Que el código final esté correcto y compile.
- **Segunda: el historial Git** -> Debe verse el árbol de commits y merges.
- **Tercera: las ramas** -> Deben aparecer las ramas locales y remotas.
- **Cuarta: el diff** -> Deben ser capaces de abrir un commit y ver qué cambió respecto al anterior.

35. Qué deben revisar en GitHub al finalizar

En GitHub deben comprobar:

- que existe la rama main,
- que están las ramas remotas creadas,
- que el historial de commits está subido,
- que el contenido final de main coincide con IntelliJ.